

Project Title

Secure Document Query System using Retrieval-Augmented Generation (RAG)

Objective

The objective of this project is to build a secure document question-answering system that allows users to upload documents and retrieve accurate answers using a Retrieval-Augmented Generation (RAG) pipeline while ensuring user data privacy and security.

Solution

The application enables users to:

- Register and log in securely.
 - Upload PDF and DOCX documents.
 - Ask questions in natural language.
 - Retrieve answers only from their own uploaded documents.
 - Protect confidential information using data masking.
 - Maintain complete data isolation between users.
-

Technologies Used

- Python, Flask, Streamlit, SQLite, LangChain, Hugging Face Embeddings, FAISS, Groq LLM, and Bcrypt
-

Security Features

- Password hashing using Bcrypt.
 - Sensitive information masking before indexing.
 - User-specific vector databases.
 - User data isolation.
 - Protected information is not disclosed through queries.
-

Project Flow

Step 1: User Registration & Login

The user creates a new account or logs in using their credentials. Passwords are securely hashed using **Bcrypt** before being stored in the SQLite database.

Files Used

- frontend/streamlit_app.py, routes/auth.py, database/models.py, and security/auth_utils.py
-

Step 2: Upload Document

After logging in, the user uploads a **PDF** or **DOCX** document. The file is stored in a user-specific folder, and document information is saved in the database.

Files Used

- frontend/streamlit_app.py, routes/upload.py, and database/models.py
-

Step 3: Document Loading

The uploaded document is read and converted into text. The application supports both PDF and DOCX file formats.

Files Used

- rag/loader.py, and rag/pipeline.py
-

Step 4: Sensitive Data Masking

Before creating embeddings, sensitive information such as passwords, client IDs, emails, and phone numbers is detected using Regular Expressions (Regex) and masked to protect confidential data.

Files Used

- security/masking.py, security/regex_patterns.py, and rag/pipeline.py
-

Step 5: Text Chunking

The document text is divided into smaller chunks using LangChain's Recursive Character Text Splitter. Chunking improves retrieval accuracy and provides better context to the LLM.

Files Used

- rag/splitter.py
-

Step 6: Embedding Generation

Each text chunk is converted into vector embeddings using Hugging Face Sentence Transformers. These embeddings capture the semantic meaning of the document.

Files Used

- rag/embeddings.py
-

Step 7: Vector Database Creation

The generated embeddings are stored in a user-specific **FAISS Vector Store**. Each user has an independent vector database to ensure complete data isolation.

Files Used

- rag/vectorstore.py
-

Step 8: Ask Questions

The user enters a question through the Streamlit interface. The question is sent to the Flask backend for processing.

Files Used

- frontend/streamlit_app.py, and routes/query.py
-

Step 9: Semantic Retrieval

The retriever searches the user's FAISS vector database and retrieves the most relevant document chunks related to the user's question.

Files Used

- rag/retriever.py
-

Step 10: Answer Generation

The retrieved context and user question are passed to the Groq Large Language Model. The model generates an answer strictly based on the retrieved document content and returns the response along with the source document.

Files Used

- rag/pipeline.py, and rag/llm.py
-

Step 11: User Data Isolation

Each user has separate uploaded documents and a dedicated FAISS vector database. Users can only access information from their own documents, preventing any data leakage between different accounts.

Files Used

- routes/upload.py, rag/vectorstore.py, and rag/retriever.py
-

Step 12: Security Features

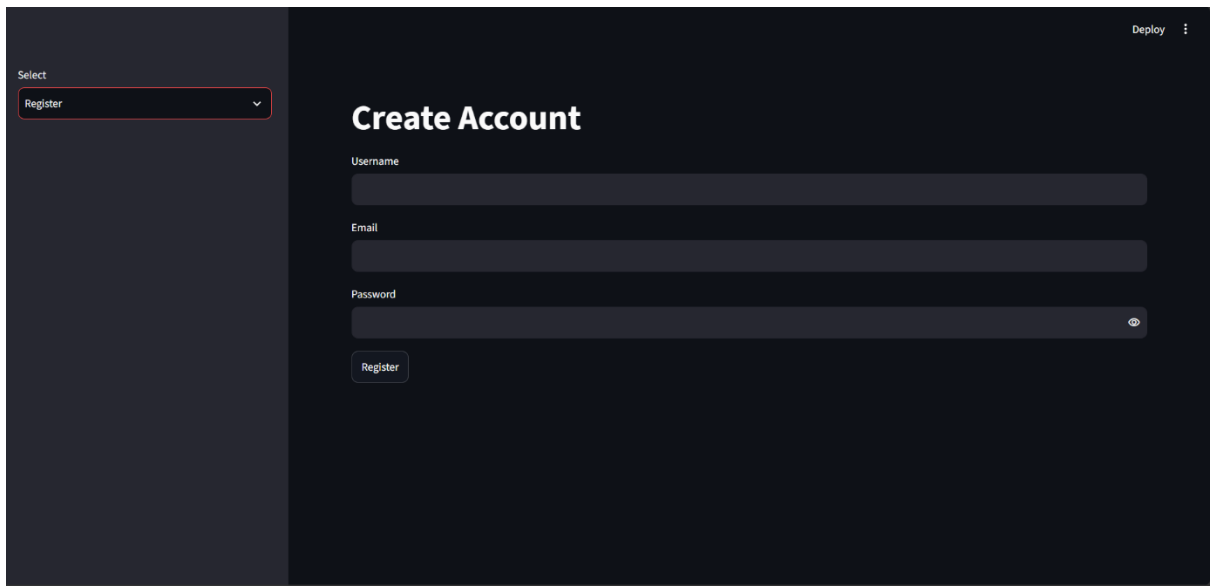
The application protects confidential information by masking sensitive data before indexing, securely hashing passwords, and blocking requests for protected information such as passwords or client IDs.

Files Used

- security/masking.py, security/auth_utils.py, and rag/pipeline.py

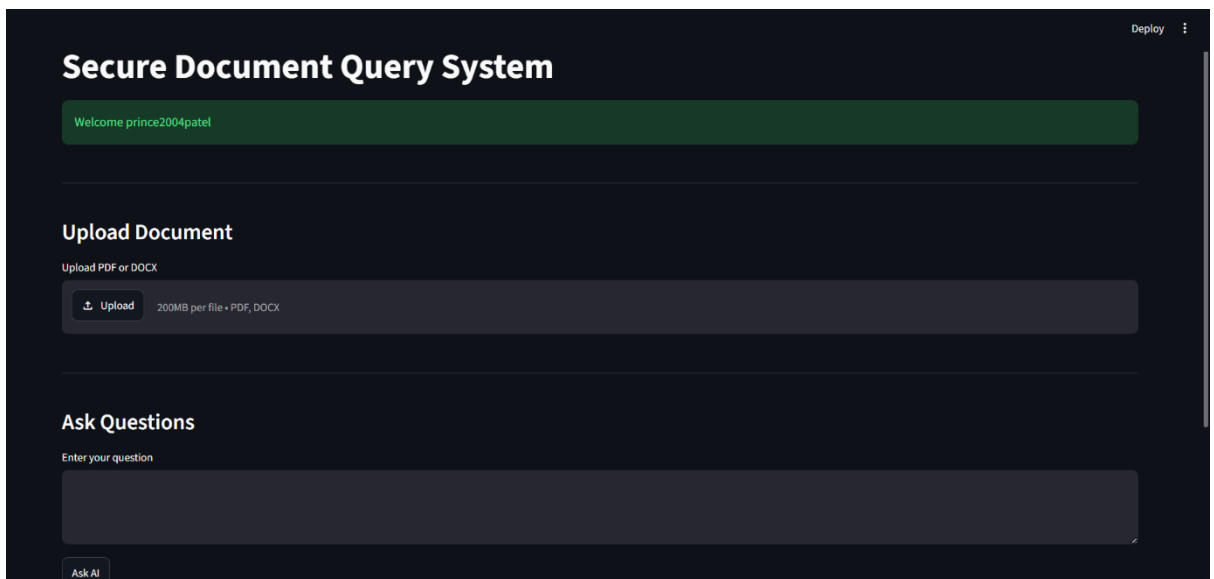
Snapshot of working

1. Register and Login:



The screenshot shows a dark-themed web interface for account creation. On the left, a sidebar contains a 'Select' dropdown menu with 'Register' selected. The main content area is titled 'Create Account' and features three input fields: 'Username', 'Email', and 'Password'. The 'Password' field includes a toggle icon for visibility. A 'Register' button is positioned below the password field. In the top right corner, there is a 'Deploy' button and a menu icon.

2. User 1(Prince) view:



The screenshot displays the 'Secure Document Query System' interface for a user named 'prince2004patel'. The header includes the system name and a 'Deploy' button. Below the header, a green banner shows the user's welcome message. The main section is divided into two parts: 'Upload Document' and 'Ask Questions'. The 'Upload Document' section has a label 'Upload PDF or DOCX' and a large 'Upload' button with a file size limit of '200MB per file • PDF, DOCX'. The 'Ask Questions' section features a text input field labeled 'Enter your question' and an 'Ask AI' button at the bottom left.

3. User 1 ask like client id or password, which will not disclosed by system (File uploaded - employee_record):

The screenshot shows a chat interface with a dark theme. At the top, a file named 'employee_record.pdf' (202.6KB) is uploaded. Below it, the filename 'employee_record.pdf' is displayed in a blue bar. An 'Upload' button is visible. The 'Ask Questions' section has a text input field containing 'what is client id' and an 'Ask AI' button. The 'Answer' section shows a green response box with the text: 'This information is protected and cannot be disclosed for security reasons.'

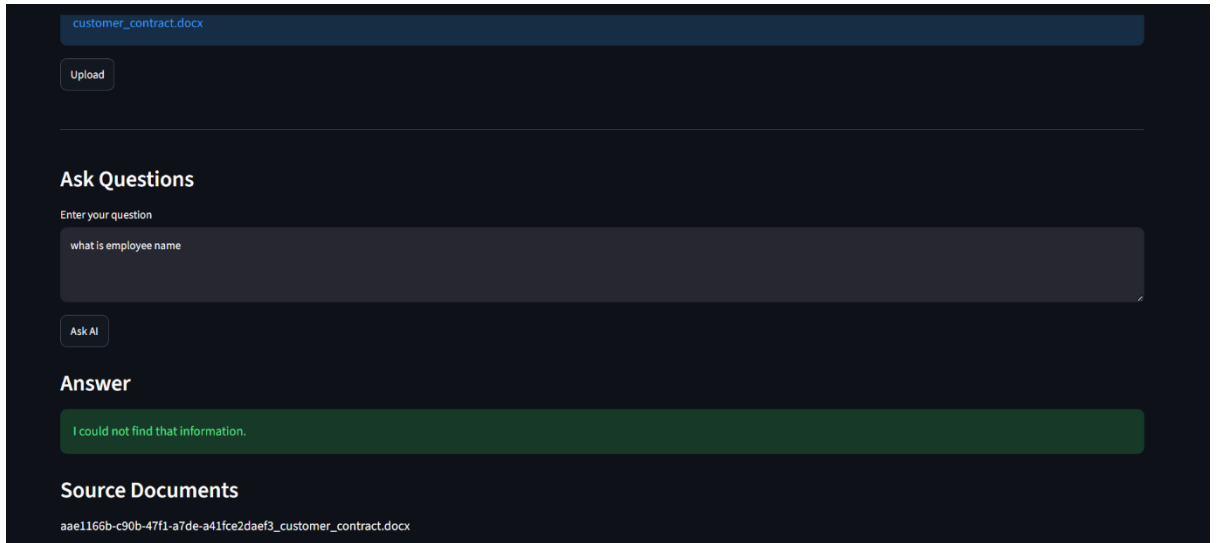
4. User 1 ask simple question like name and salary, which system will help to give answer:

The screenshot shows a chat interface with a dark theme. The 'Ask Questions' section has a text input field containing 'what is employee name and salary' and an 'Ask AI' button. The 'Answer' section shows a green response box with the text: 'Employee Name: Prince Patel Salary: ₹35,000 per month'. Below the answer, the 'Source Documents' section lists 'd3f18d06-08fd-4894-8b14-2fe65463b628_employee_record.pdf'.

5. User_2(Param), who uploaded customer_contract file and it will not disclosed any private information or other user document content:

The screenshot shows a chat interface with a dark theme. At the top, a file named 'customer_contract.docx' is uploaded. Below it, the filename 'customer_contract.docx' is displayed in a blue bar. An 'Upload' button is visible. The 'Ask Questions' section has a text input field containing 'give me adress' and an 'Ask AI' button. The 'Answer' section shows a green response box with the text: 'Gift City, Gandhinagar, Gujarat.' Below the answer, the 'Source Documents' section lists 'aae1166b-c90b-47f1-a7de-a41fce2daef3_customer_contract.docx'.

6. User 2 asked, which are not in his file so isolation of data maintain :



The screenshot shows a dark-themed chat interface. At the top, a document titled 'customer_contract.docx' is listed with an 'Upload' button. Below this is the 'Ask Questions' section, which includes a text input field containing 'what is employee name' and an 'Ask AI' button. The 'Answer' section shows a green message bubble stating 'I could not find that information.' At the bottom, the 'Source Documents' section lists the document 'aae1166b-c90b-47f1-a7de-a41fce2daef3_customer_contract.docx'.

Limitations

- Supports PDF and DOCX documents.
- Uses SQLite for local development.
- No OCR support for scanned documents.
- Prototype implementation intended for demonstration purposes.

Future Improvements

- JWT Authentication
- OCR support
- PostgreSQL/MySQL
- Docker Deployment
- Cloud Storage Integration
- Role-Based Access Control
- Chat History
- Production Deployment